AWA ASSET MANAGEMENT · INTERNAL ARCHITECTURE NOTE

DM Onboarding & Anand Rathi Integration

What the client onboarding process involves, and how to use the broker API without duplicating work.

BROKER	KYC PORTAL	AWA BACKEND
Anand Rathi · RMCODE 607138	DigiAvatar	Node.js / Neon PostgreSQL / Render

STORAGE

Cloudinary · Resend Email

PART 1

What client onboarding actually involves

Looking at the Partner Enrollment spreadsheet, each Distribution Manager (DM) record requires three categories of data. This is your source of truth for what the form must collect.

IDENTITY & CATEGORY

- Distribution Number (DM000001)
- Category (CA / LIC / Direct)
- Distribution Name
- Secondary Name

PRODUCTS & DOCUMENTS

- Products: Land, PMS, Mutual Fund
- PAN Card
- Aadhar Card
- Cancelled Cheque

NOMINEE & BANK

- Nominee Name
- Nominee Relationship
- Bank Name
- Account No. & IFSC

PART 2

The API split strategy

You collect. They validate. You store the result.

Don't rebuild what the broker already does — collect once, hand off, receive status back.

What AWA's API owns — keep this

- ✓ Accept form data (name, mobile, email, nominee, docs)
- ✓ Store everything in Neon PostgreSQL
- ✓ Generate the DM ID (DM000001 format)
- ✓ Send the welcome email via Resend
- ✓ Expose `/api/partner/:id/status`

What Anand Rathi owns — don't rebuild

- PAN verification against NSDL
- Aadhar verification against UIDAI
- Bank account penny drop / IFSC validation
- CKYC, FATCA, IPV compliance checks
- The DigiAvatar onboarding portal UI

PART 3

The two integration points to build

01 · Pre-fill Handoff

Redirect to DigiAvatar

When Step 3 (docs uploaded) is complete, redirect the DM to DigiAvatar with their data pre-filled as URL parameters. This avoids the DM typing the same info twice.

`URL parameter pre-fill`

02 · Webhook Receiver

KYC status callback

One endpoint that Anand Rathi calls when KYC status changes. It updates your DB and triggers a follow-up email. No polling needed.

`POST /api/kyc/webhook`

One thing to confirm with Anand Rathi RM right now

→ Ask for the exact URL parameter names DigiAvatar accepts for pre-fill (name, PAN, mobile, etc.)

— that's the only blocker for the pre-fill handoff. Everything else can be built today.

- → Ask what the webhook payload looks like — exact field names for partner_id, ucc_code, status.
- → Ask whether they send a secret/signature header (for webhook verification against spoofing).
- → Ask what the possible status values are: approved / rejected / pending / under_review?

PART 4

What happens after KYC approval

Three things happen simultaneously the moment Anand Rathi approves — on their side, on your system, and for the DM. One webhook handler on your end kicks off the entire AWA column automatically.

ANAND RATHI SIDE	AWA SYSTEM	DM EXPERIENCE
UCC generated Unique client code issued	**DB status update** kyc_status = approved	**Approval notification** Email + SMS confirmation
RMCODE linked 607138 attached to account	**Activation email** Via Resend — DM ID + UCC	**DM ID + UCC received** Their two key identifiers
Account activated Ready to place orders	**Admin dashboard** DM marked active, auditable	**Can refer clients** Share referral link
Login credentials Broker portal access sent	**Referral link generated** Unique DM onboarding URL	**Products enabled** Land, PMS, Mutual Funds
	DM portal unlocked View clients, commissions	

PART 5

The webhook handler — build this first

A single endpoint in your existing Express backend handles the entire post-approval flow. Add this to your routes:

```
// routes/kyc.js — add this one route
router.post('/api/kyc/webhook', async (req, res) => {
  const { partner_id, ucc_code, status } = req.body;

  // Handle non-approved statuses first
  if (status !== 'approved') {
    await db.query(
      `UPDATE partners SET kyc_status = $1 WHERE partner_id = $2`,
      [status, partner_id]
    );
    return res.json({ success: true });
  }

  // 1. Update DB with approval + UCC code
  await db.query(
    `UPDATE partners
     SET kyc_status = 'approved', ucc_code = $1, activated_at = NOW()
     WHERE partner_id = $2`,
    [ucc_code, partner_id]
  );

  // 2. Fetch partner details for email
  const { rows } = await db.query(
    `SELECT * FROM partners WHERE partner_id = $1`,
    [partner_id]
  );
  const partner = rows[0];

  // 3. Generate unique referral URL
  const referralUrl =
    `https://awaasset.com/partner-enrollment?ref=${partner.partner_id}`;

  // 4. Send activation email via Resend
  await sendActivationEmail(partner.email, {
    name:        partner.name,
    dmId:        partner.partner_id,
    uccCode:     ucc_code,
    referralUrl
  });

  res.json({ success: true });
});
```

This is a one-afternoon build — once you have four things from the RM

→ Webhook payload field names (partner_id, ucc_code, status — or their equivalents)

Secret header for webhook signature verification

→

→ Possible status values (approved / rejected / pending / under_review)

→ DigiAvatar pre-fill URL parameter names (name, PAN, mobile, etc.)